

Générer un Fichier XML avec Python

Guide complet et pratique utilisant la bibliothèque native `xml.etree.ElementTree`

Principe clé : Le module standard `xml.etree.ElementTree` permet d'interagir avec les structures XML sous forme d'un arbre d'éléments rattachés les uns aux autres (racine, enfants, attributs, texte).

1 Étape 1 : Structurer les données source

Avant d'écrire du XML, nous préparons nos données Python sous forme d'une liste de dictionnaires. C'est le format typique renvoyé par une base de données ou un script d'analyse.

```
# Données brutes récoltées dans le programme Python donnees = [ {"id": "101", "nom": "Clavier mécanique", "prix": "59.90"}, {"id": "102", "nom": "Souris sans fil", "prix": "24.99"}, {"id": "103", "nom": "Écran 27 pouces", "prix": "199.00"} ]
```

2 Étape 2 : Créer l'arborescence XML

Nous importons le module, créons le nœud racine (ex: `<catalogue>`), puis nous le peuplons en bouclant sur la liste.

```
import xml.etree.ElementTree as ET # 1. Création de la balise racine principale
racine = ET.Element("catalogue") # 2. Boucle sur les données pour alimenter l'arbre XML
for produit in donnees: # Création d'un élément enfant

    rattaché à la racine
    elem_produit = ET.SubElement(racine, "produit", id=produit["id"]) # Ajout de la
    sous-balise

    Texte
    elem_nom = ET.SubElement(elem_produit, "nom") elem_nom.text = produit["nom"] # Ajout
    de la sous-balise

    Texte
    elem_prix = ET.SubElement(elem_produit, "prix") elem_prix.text = produit["prix"]
```

3 Étape 3 : Formater et enregistrer le fichier

Par défaut, Python génère un XML sur une seule ligne. L'instruction `ET.indent()` permet d'ajouter une indentation propre avant d'écrire le fichier sur le disque.

```
# 3. Encapsuler la racine dans un objet ElementTree arbre = ET.ElementTree(racine) #
4. Appliquer un formatage lisible (sauts de ligne + indentation) ET.indent(arbre, space=" ")
# 5. Enregistrer le document XML sur le disque arbre.write( "catalogue.xml", encoding="utf-8", xml_declaration=True )
```

Résultat obtenu : catalogue.xml

```
<?xml version='1.0' encoding='utf-8'?> <catalogue> <produit id="101"> <nom>Clavier
mécanique</nom> <prix>59.90</prix> </produit> <produit id="102"> <nom>Souris sans fil</nom>
<prix>24.99</prix> </produit> </catalogue>
```

Synthèse des Fonctions Essentielles

Méthode / Attribut	Description	Exemple d'utilisation
<code>ET.Element(tag)</code>	Crée la balise racine du document.	<code>r = ET.Element("racine")</code>
<code>ET.SubElement(parent, tag)</code>	Crée et rattache un nœud enfant au parent.	<code>e = ET.SubElement(r, "item")</code>
<code>element.text = "valeur"</code>	Définit le contenu texte entre les balises.	<code>e.text = "Mon texte"</code>
<code>ET.indent(tree, space)</code>	Formate l'arbre avec des retours à la ligne.	<code>ET.indent(arbre, space=" ")</code>
<code>tree.write(nom, ...)</code>	Écrit le fichier XML final sur le disque.	<code>arbre.write("out.xml")</code>



Attribut vs Balise

Utilisez des **attributs** (ex: `id="101"`) pour les métadonnées courtes ou identifiants uniques, et des **balises enfants** pour le contenu principal ou descriptif.



Gestion de l'encodage

Spécifiez toujours `encoding="utf-8"` et `xml_declaration=True` lors de l'appel à `write()` pour garantir le bon affichage des caractères accentués.